Graphical Analytics

Repository root: ***Graphical Analytics using Python & MySQL***
Primary entry point: `main.py`

Table of Contents

Executive Summary

This document provides a deep, end-to-end technical walkthrough of Graphical Analytics tools written in Python using MySQL Database with entry-point as `main.py`. The documentation explains the runtime flow, dependencies, environment assumptions, and extension points. It also embeds code with syntax highlighting and concludes with a comprehensive, practical guide to **all Python reserved keywords**, each with a concise explanation.

main.py

Source Code

```python
# main.py

import database_handler as database_handler
import visual_handler as visual_handler

database_handler.enter_mysql()
visual_handler.run_GUI()
```

What the code does

1. Imports a module named `database_handler` and aliases it to the same name.
2. Imports a module named `visual_handler` and aliases it to the same name.
3. Calls a function `enter_mysql()` from `database_handler` — presumably to establish a MySQL connection and/or prepare application data.
4. Calls a function `run_GUI()` from `visual_handler` — presumably to start a GUI loop or render a visualization.

Assumptions:

- `database_handler.py` and `visual_handler.py` exist in the same package (or along `PYTHONPATH`).
- `database_handler.enter_mysql()` performs connection/initialization and raises exceptions on failure.
- `visual_handler.run_GUI()` blocks the main thread until the GUI is closed (typical for GUI event loops), or returns when initialization fails.

Runtime Architecture & Control Flow

High-level Flow

```
main.py
   ├─ import database_handler
   ├─ import visual_handler
```

```
   ├─ database_handler.enter_mysql()   # DB setup (credentials, connection,
migrations)
   └─ visual_handler.run_GUI()         # GUI init + event loop (rendering,
input handling)
```

Sequence Diagram

```
sequenceDiagram
    autonumber
    participant App as main.py
    participant DB as database_handler
    participant UI as visual_handler

    App->>DB: enter_mysql()
    DB-->>App: connection handle / ready state
    App->>UI: run_GUI(connection)
    UI-->>App: event loop exit / termination
```

If `visual_handler.run_GUI()` needs database access, pass a connection/session (shown above) or inject a service object rather than using implicit globals.

Environment & Dependencies

Python Runtime

Recommended: Python 3.10+ (to benefit from match/case, modern typing, better error messages). The code itself is compatible with 3.6+.

Third-party Libraries

- **MySQL driver**: likely one of `mysqlclient`, `PyMySQL`, or `mysql-connector-python` inside `database_handler`.
- **GUI**: possibly `tkinter`, `PyQt5/PyQt6`, `PySide6`, or `matplotlib` for plotting inside `visual_handler`.

database_handler.py

Source Code

```
# Requires MqSQL connector
import mysql.connector as conn
import setup as setup
import sys # to exit the program

conn_obj = "" # this is used to create the connection to mysql via enter_mysql()

def enter_mysql():
   global conn_obj
```

```
    correct_credentials = False
    while correct_credentials != True:
        try:
            user_name = str(input("Please enter your MySQL username: "))
            user_password = str(input("Please enter your MySQL password: "))
            user_host = str(input("Please enter MySQL host: ('localhost' if using localhost): "))
            user_port = str(input("Please enter MySQL port: ('n' if using localhost): "))
            print("Attempting connection...")

            if user_port == 'n':
                conn_obj = conn.connect(host = user_host, user = user_name, passwd = user_password)
            else:
                conn_obj = conn.connect(host = user_host, port = user_port, user = user_name, passwd = user_
password)

            correct_credentials = True
        except Exception as e:
            exit_program = input("There was a mistake with the username, password or port: " + str(e) + "\nPl
ease any key to try again (or 'q' to quit) \n")
            if exit_program == "q":
                sys.exit(1) # exits program


    if conn_obj.is_connected() == True:
        print("Connected successfully")

    cursor = conn_obj.cursor()
    setup.initialise_tables(cursor)
    cursor.close()

def import_items(values):

    # values is a list of tuples; each tuple describes an item in the format: (item_name,item_cost,item_gst,ite
m_discount,item_final_cost,item_margin,item_stock,item_manufacturer_name,item_manufacturer_incharge,
item_manufacturer_contact_no)
    command = """INSERT INTO inventory (item_name,item_cost,item_gst,item_discount,item_final_cost,ite
m_margin,item_stock,item_restock_value,item_manufacturer_name,item_manufacturer_incharge,item_man
ufacturer_contact_no)
                VALUES
            """

    for i in values:
        command += str(i) + ","

    command = command[:-1] + ";" # as last element ends with colon not comma

    cursor = conn_obj.cursor()
```

```
    cursor.execute("USE main_database")
    cursor.execute("DELETE FROM inventory")
    cursor.execute("ALTER TABLE inventory AUTO_INCREMENT = 1")
    cursor.execute(command)
    cursor.execute("commit")
    cursor.close()

def import_orders(values):

    # values is a list of tuples; each tuple describes an order in the format: (order_item_name,order_initial_c
ost,order_gst,order_discount,order_final_cost,order_quantity,order_customer_name,order_customer_contac
t_no)
    command = """INSERT INTO orders (order_item_name,order_date,order_initial_cost,order_gst,order_disc
ount,order_final_cost,order_quantity,order_customer_name,order_customer_contact_no)
                VALUES
              """

    for i in values:
        command += str(i) + ","

    command = command[:-1] + ";" # as last element ends with colon not comma

    cursor = conn_obj.cursor()
    cursor.execute("USE main_database")
    cursor.execute("DELETE FROM orders")
    cursor.execute("ALTER TABLE orders AUTO_INCREMENT = 1")
    cursor.execute(command)
    cursor.execute("commit")
    cursor.close()

def retrieve_via_sql_query(sql_select, sql_from, sql_where = "", sql_group_by = ""):
    command = "SELECT " + sql_select + "\n"

    if sql_where == "" and sql_group_by == "":
        # sql command with select and from only
        command += "FROM " + sql_from + ";" # if no where clause specified
    elif sql_where != "" and sql_group_by == "":
        #sql command with select, from and where
        command += "FROM " + sql_from + "\n"
        command += "WHERE " + sql_where + ";"
    else:
        #sql command with select, from and group by
        command += "FROM " + sql_from + "\n"
        command += "GROUP BY " + sql_group_by + ";"



    cursor = conn_obj.cursor()
```

```
    cursor.execute(command)

    data = cursor.fetchall()

    cursor.execute("commit")
    cursor.close()

    return(data)

def retrieve_headers(sql_from):
    command = "DESC " + sql_from

    cursor = conn_obj.cursor()
    cursor.execute(command)

    data = cursor.fetchall()

    cursor.execute("commit")
    cursor.close()

    return(data)
```

Module Overview

- **Name:** `database_handler`
- **Purpose:** Provides MySQL database connection establishment, initialization, data insertion into tables (`inventory` and `orders`), and retrieval operations (data rows and headers).
- **Global State:**
    - `conn_obj`: Initially an empty string, reassigned to hold the active MySQL connection object.

Function Documentation

enter_mysql()

- **Input:** User provides MySQL username, password, host, and port through `input()`.
- **Behavior:** Attempts to connect to the MySQL server. If connection fails, user can retry or quit (`sys.exit(1)`).
- **Post-Connection:**
    - Prints confirmation message if `is_connected()` returns `True`.
    - Initializes tables by invoking `setup.initialise_tables(cursor)`.
- **Side Effects:**
    - Reads from standard input, writes to standard output.
    - Sets global `conn_obj` to a connection.

import_items(values)

- **Input:** `values` → list of tuples describing inventory items.
- **Behavior:**
  - Builds an SQL `INSERT INTO inventory ...` statement with all tuples.
  - Clears existing table contents (`DELETE FROM inventory`).
  - Resets auto-increment counter.
  - Inserts new values and commits.
- **Side Effects:** Executes SQL altering table contents.

import_orders(values)

- **Input:** `values` → list of tuples describing orders.
- **Behavior:**
  - Builds SQL `INSERT INTO orders ...` statement.
  - Clears existing table contents and resets auto-increment.
  - Inserts new order rows and commits.
- **Side Effects:** Executes SQL altering table contents.

retrieve_via_sql_query(sql_select, sql_from, sql_where = "", sql_group_by = "")

- **Input:** Strings specifying `SELECT`, `FROM`, optional `WHERE`, and optional `GROUP BY` parts.
- **Behavior:** Dynamically constructs an SQL query and executes it.
- **Output:** Returns fetched rows (list of tuples).
- **Side Effects:** Executes SQL on the connected database.

retrieve_headers(sql_from)

- **Input:** Table name as a string.
- **Behavior:** Executes `DESC <table>` to fetch schema description.
- **Output:** Returns fetched description rows (list of tuples).
- **Side Effects:** Executes SQL on the connected database.

SQL Operations Summary

- Database schema assumed: `main_database`.
- Tables: `inventory`, `orders`.
- Operations include `INSERT`, `DELETE`, `ALTER TABLE ... AUTO_INCREMENT`, `DESC`, `SELECT`, `USE`, and `commit`.

Python Reserved Keywords

Literals

- `True`, `False`, `None`

```
flag = True
empty = None
```

Control Flow

- `if`, `elif`, `else`

Boolean Logic

- `and`, `or`, `not`

Membership and Identity

- `in`, `is`

Functions and Generators

- `def`, `return`, `yield`

Pattern Matching (3.10+)

- `match`, `case`

Exceptions

- `try`, `except`, `finally`, `raise`, `assert`

Imports and Aliasing

- `import`, `from`, `as`

Context Management

- `with`

Classes and Lambdas

- `class`, `lambda`

Async Programming

- `async`, `await`

Full Keyword List (Python 3.12)

`False`, `None`, `True`, `and`, `as`, `assert`, `async`, `await`, `break`, `class`, `continue`, `def`, `del`, `elif`, `else`, `except`, `finally`, `for`, `from`, `global`, `if`, `import`, `in`, `is`, `lambda`, `match`, `nonlocal`, `not`, `or`, `pass`, `raise`, `return`, `try`, `while`, `with`, `yield`, `case`.

setup.py

Source Code

```
# Only run once on initialisation

def initialise_tables(cursor):
  # Working under databases called main_database and plot_database
  cursor.execute("CREATE DATABASE IF NOT EXISTS main_database")

  cursor.execute("USE main_database")
  # margin, discount is in percentages and number is in varchar to handle leading zeros
  cursor.execute("DROP TABLE IF EXISTS inventory;") # Handles setup being run more than once
  cursor.execute("""CREATE TABLE main_database.inventory
                (
                    item_id int(10) not null AUTO_INCREMENT,
                    item_name varchar(20) not null,
                    item_cost float(8,2) not null,
                    item_gst float(8,2) not null,
                    item_discount float(5,2) not null,
                    item_final_cost float(8,2) not null,
                    item_margin float(5,2) not null,
                    item_stock int(10) not null,
                    item_restock_value int(10) not null,
                    item_manufacturer_name varchar(40) not null,
                    item_manufacturer_incharge varchar(20) not null,
                    item_manufacturer_contact_no varchar(10) not null,
                    PRIMARY KEY (item_id)
                );""")

  cursor.execute("""INSERT INTO inventory (item_name,item_cost,item_gst,item_discount,item_final_cost,ite
m_margin,item_stock,item_restock_value,item_manufacturer_name,item_manufacturer_incharge,item_man
ufacturer_contact_no)
                VALUES
                ("Apples",139.00,10.00,5.00,122.55,50.00,500,50,"APPLE","Tim Cook","0999666333"),
                ("Raspberry",139.00,10.00,5.00,122.55,50.00,500,50,"RASPBERRY PI","Eben Upton","0111222333"
),
                ("Orange",139.00,10.00,5.00,122.55,50.00,500,50,"ORANGE PI","Zhao Yifan","0555666777"),
                ("Bannana",139.00,10.00,5.00,122.55,50.00,500,50,"BANNANA PI","U.N. Owen","0333444555"),
                ("Potato",139.00,10.00,5.00,122.55,50.00,500,50,"LE POTATO LIBRE","W.H. Onos","0777888999");
                """)
  cursor.execute('commit')


  # discount is in percentages and number is in varchar to handle leading zeros
  cursor.execute("DROP TABLE IF EXISTS orders;") # Handles setup being run more than once
  cursor.execute("""CREATE TABLE main_database.orders
                (
```

```
                order_id int(10) not null AUTO_INCREMENT,
                order_item_name varchar(20) not null,
                order_date date not null,
                order_initial_cost float(8,2) not null,
                order_gst float(8,2) not null,
                order_discount float(5,2) not null,
                order_final_cost float(8,2) not null,
                order_quantity int(10) not null,
                order_customer_name varchar(20) not null,
                order_customer_contact_no varchar(10),
                PRIMARY KEY (order_id)
            );""")

 cursor.execute("""INSERT INTO orders (order_item_name,order_date,order_initial_cost,order_gst,order_dis
count,order_final_cost,order_quantity,order_customer_name,order_customer_contact_no)
            VALUES
            ("Apples","2025-04-08",139.00,10.00,5.00,122.55,1,"Tom Bakes","1999666333"),
            ("Raspberry","2025-04-08",139.00,10.00,5.00,122.55,1,"Nebe Downton","1111222333"),
            ("Orange","2025-04-08",139.00,10.00,5.00,122.55,1,"Zhao Yistan","1555666777"),
            ("Bannana","2025-04-08",139.00,10.00,5.00,122.55,1,"K.N. Owen","1333444555"),
            ("Potato","2025-04-08",139.00,10.00,5.00,122.55,1,"W.E. Noe","1777888999");
            """)
 cursor.execute('commit')
```

Module Overview

- **Name:** `setup`
- **Purpose:** Defines `initialise_tables`, which ensures the database `main_database` and its tables (`inventory`, `orders`) exist, and populates them with default rows.

Function Documentation

initialise_tables(cursor)

- **Input:**
  - `cursor`: Database cursor object.
- **Behavior:**
  1. Creates `main_database` if it does not exist.
  2. Switches to `main_database`.
  3. Drops and recreates `inventory` with specified columns and constraints.
  4. Inserts multiple default rows into `inventory`.
  5. Commits transaction.
  6. Drops and recreates `orders` with specified columns and constraints.
  7. Inserts multiple default rows into `orders`.
  8. Commits transaction.

- **Output:** None (no explicit return).
- **Side Effects:** Alters database schema and inserts rows.

SQL Operations Summary

- **Database:** `main_database`
- **Tables:** `inventory`, `orders`
- **Operations:** `CREATE DATABASE`, `USE`, `DROP TABLE`, `CREATE TABLE`, `INSERT`, `commit`

Python Reserved Keywords

Literals

- `True`, `False`, `None`

Control Flow

- `if`, `elif`, `else`

Boolean Logic

- `and`, `or`, `not`

Membership and Identity

- `in`, `is`

Functions and Generators

- `def`, `return`, `yield`

Pattern Matching (Python 3.10+)

- `match`, `case`

Exceptions

- `try`, `except`, `finally`, `raise`, `assert`

Imports and Aliasing

- `import`, `from`, `as`

Namespace Control

- `global`, `nonlocal`, `del`

Context Management

- `with`

Classes and Lambdas

- `class`, `lambda`

Async Programming

- `async`, `await`

Full Keyword List (Python 3.12)

`False`, `None`, `True`, `and`, `as`, `assert`, `async`, `await`, `break`, `class`, `continue`, `def`, `del`, `elif`, `else`, `except`, `finally`, `for`, `from`, `global`, `if`, `import`, `in`, `is`, `lambda`, `match`, `nonlocal`, `not`, `or`, `pass`, `raise`, `return`, `try`, `while`, `with`, `yield`, `case`.

visual_handler.py

Source Code

```
import tkinter as tk # for general GUI
from tkinter import ttk # for acess to the scrollbar and treeview widgets
from tkinter.filedialog import askopenfilename # for inputting the CSV file
from numpy import genfromtxt # for handling the CSV
import database_handler as database_handler # for handling the database
import matplotlib.pyplot as plt # for plotting
from ttkbootstrap.widgets import Button, Treeview, Label # for creating come stylised widgets
from ttkbootstrap import Style # for overall colour pallate of the widgets
import datetime # for retreiving date
import sys # for quitting the application


def run_GUI():
  # all window definitions
  root = tk.Tk()
  root.title("Python Inventory Analytics")
  root.geometry("660x365")
  root.protocol('WM_DELETE_WINDOW', sys.exit)


  home_view = tk.Frame(root)
  home_view.grid(row = 1, column = 1, sticky = 'news')

  modelling_view = tk.Frame(root)
  modelling_view.columnconfigure(0,weight = 1)
  modelling_view.rowconfigure(1, weight = 1)
  modelling_view.grid(row = 1, column = 1, sticky = 'news')

  # setting style
  style = Style("solar")

  # functions for home view
  def populate_data():
```

```
  # removing response message if any
  response_message.set("")

  # inventory viewer
  inventory_data = database_handler.retrieve_via_sql_query("item_id,item_name,item_cost,item_final_cost,i
tem_stock","inventory")
  inventory_viewer = Treeview(inventory_tab,
                        columns = ("item_id","item_name","item_cost","item_final_cost","item_stock"),
                        show = 'headings',
                        height = 13,
                        bootstyle = 'success'
                        )
  inventory_viewer.grid(row = 1,
                    column = 0,
                    sticky = "e"
                    )

  # creating the scrollbar
  scrollbar = ttk.Scrollbar(inventory_tab, orient = "vertical", command = inventory_viewer.yview)
  scrollbar.grid(row = 1,
                column = 1,
                sticky = "nsew"
                )
  inventory_viewer.configure(yscrollcommand = scrollbar.set)

  # initialising columns
  inventory_viewer.column("item_id", anchor = "center", width = 45)
  inventory_viewer.heading('item_id', text = 'S.No')

  inventory_viewer.column("item_name", anchor = "center", width = 75)
  inventory_viewer.heading('item_name', text = 'Name')

  inventory_viewer.column("item_cost", anchor = "center", width = 55)
  inventory_viewer.heading('item_cost', text = 'Cost')

  inventory_viewer.column("item_final_cost", anchor = "center", width = 55)
  inventory_viewer.heading('item_final_cost', text = 'Total')

  inventory_viewer.column("item_stock", anchor = "center", width = 50)
  inventory_viewer.heading('item_stock', text = 'Stock')

  inventory_viewer.grid(row = 1, column = 0)

  # insert values into inventory_viewer
  for i in inventory_data:
    inventory_viewer.insert(parent = '', index = tk.END, values = i)

  # orders viewer code
```

```
  orders_data = database_handler.retrieve_via_sql_query("order_id,order_item_name,order_customer_nam
e,order_final_cost,order_quantity","orders")

  orders_viewer = Treeview(orders_tab,
                      columns = ("order_id","order_item_name","order_customer_name","order_final_cost","or
der_quantity"),
                      show = 'headings',
                      height = 13,
                      bootstyle = 'success'
                      )
  orders_viewer.grid(row = 1,
                  column = 0,
                  sticky = "e"
                  )

  # creating the scrollbar
  scrollbar = ttk.Scrollbar(orders_tab, orient = "vertical", command = orders_viewer.yview)
  scrollbar.grid(row = 1,
                column = 1,
                sticky = "nsew"
                )
  orders_viewer.configure(yscrollcommand = scrollbar.set)

  # initialising columns
  orders_viewer.column("order_id", anchor = "center", width = 45)
  orders_viewer.heading('order_id', text = 'S.No')

  orders_viewer.column("order_item_name", anchor = "center", width = 75)
  orders_viewer.heading('order_item_name', text = 'Item')

  orders_viewer.column("order_customer_name", anchor = "center", width = 60)
  orders_viewer.heading('order_customer_name', text = 'Name')

  orders_viewer.column("order_final_cost", anchor = "center", width = 55)
  orders_viewer.heading('order_final_cost', text = 'Total')

  orders_viewer.column("order_quantity", anchor = "center", width = 45)
  orders_viewer.heading('order_quantity', text = 'Amt')

  orders_viewer.grid(row = 1, column = 0)

  # insert values into orders_viewer
  for i in orders_data:
    orders_viewer.insert(parent = '', index = tk.END, values = i)

 def import_database():
   if full_inventory_path.get() !=  "" and full_orders_path.get() !=  "":
     try:
```

```
        items = genfromtxt(full_inventory_path.get(), delimiter = ",", dtype = None, skip_header = 1, encodin
g = "utf8")
        database_handler.import_items(items)
      except Exception as e:
        # error with csv file
        print(e)
        response_message.set("Import Unsuccessful; Please check your CSV")
        spacer.config(fg = "red")
        return None

      try:
        orders = genfromtxt(full_orders_path.get(), delimiter = ",", dtype = None, skip_header = 1, encoding
= "utf8")
        database_handler.import_orders(orders)
      except:
        # error with csv file
        response_message.set("Import Unsuccessful; Please check your CSV")
        spacer.config(fg = "red")
        return None

      # if function reaches here, the code was sucessfull
      response_message.set("Import Successful; Please Refresh")
      spacer.config(fg = "green")
    else:
      response_message.set("Import Unsuccessful; Please try again")
      spacer.config(fg = "red")

  def set_inventory_path():
    full_inventory_path.set(tk.filedialog.askopenfilename())
    temp = full_inventory_path.get()
    inventory_path.set("Inventory path: \n" + temp[:20] + "...")

  def set_orders_path():
    full_orders_path.set(tk.filedialog.askopenfilename())
    temp = full_inventory_path.get()
    orders_path.set("Orders path: \n" + temp[:20] + "...")

  def refresh():
    populate_data()
    populate_inventory_low_stocks_data()
    populate_orders_highest_spends_data()
    set_inventory_response_message("Database refreshed")

  def switch_to_modelling_view():
    modelling_view.tkraise()
  # home view GUI
  database_label = tk.Label(home_view,text = "□□ ▪ ▣▓ ▩ ▒ Database Viewer ▒ ▩ ▓▣ ▪ □□", relief =
"ridge", font = "TkFixedFont")
```

```
  database_label.grid(row = 0,
                      column = 0,
                      padx = 10,
                      pady = 10,
                      sticky = "nesw"
                      )

  other_functions_label = tk.Label(home_view, text = ' ▭▣▓ ▩ ▒ Other Functions ▒ ▩ ▓▣▭ ', relief = "
ridge", font = "TkFixedFont")
  other_functions_label.grid(row = 0,
                             column = 1,
                             sticky = "ew"
                             )

  button_frame = tk.Frame(home_view)
  button_frame.grid(row = 1,
                    column = 1,
                    sticky = "nsew"
                    )

  first_row_frame = tk.Frame(button_frame)
  first_row_frame.grid(row = 0,
                       columnspan = 3,
                       pady = 5,
                       sticky = "nsew"
                       )

  modelling_view_button = Button(first_row_frame,
                                 text = "▰▱▰▱▰▰▱▰\n 📊 Modelling \n Viewport \n ▰▱▰▱▰▰▱▰",
                                 command = switch_to_modelling_view,
                                 bootstyle = "primary-outline"
                                 )

  modelling_view_button.grid(row = 0,
                             column = 0,
                             pady = 5,
                             sticky = "e"
                             )

  refresh_database_button = Button(first_row_frame,
                                   text = "▰▱▰▱▰▰▱▰ \n ↻ Refresh \n  Database \n ▰▱▰▱▰▰▱▰",
                                   command = refresh,
                                   bootstyle = "warning-outline"
                                   )

  refresh_database_button.grid(row = 0,
                               column = 1,
                               padx = 5,
```

```
                      pady = 5,
                      sticky = "nsew"
                      )


full_inventory_path = tk.StringVar()
full_orders_path = tk.StringVar()

import_database_button = Button(first_row_frame,
                        text = "▰▱▰▱▰▰▱▰ \n 🗎 Import \n Database \n ▰▱▰▱▰▰▱▰",
                        command = import_database,
                        bootstyle = "success-outline"
                        )

import_database_button.grid(row = 0,
                      column = 2
                      )

inventory_path = tk.StringVar()
inventory_path_button = tk.Button(button_frame,
                          textvariable = inventory_path,
                          command = set_inventory_path,
                          font = "TkFixedFont",
                          height = 3
                          )
inventory_path.set("Set inventory path")

inventory_path_button.grid(row = 1,
                     columnspan = 3,
                     sticky = "nsew"
                     )

orders_path = tk.StringVar()
orders_path_button = tk.Button(button_frame,
                         textvariable = orders_path,
                         command = set_orders_path,
                         font = "TkFixedFont",
                         height = 3
                         )
orders_path.set("Set orders path")

orders_path_button.grid(row = 2,
                    columnspan = 3,
                    sticky = "nsew"
                    )

response_message = tk.StringVar()
spacer = tk.Label(button_frame,textvariable = response_message, relief = "raised")
```

```
  spacer.grid(row = 3,
              columnspan = 3,
              pady = 5,
              sticky = "nsew"
              )
  response_message.set("")

  exit_button = tk.Button(button_frame,
                      text = "▰▱▰▱▰▱▰▱▰▱▰▱▰▱▰▱▰▱ \nQuit Python-Inventory-
Analytics\n ▰▱▰▱▰▱▰▱▰▱▰▱▰▱▰▱▰▱",
                      command = sys.exit,
                      font = "TkFixedFont",
                      height = 2
                      )
  exit_button.grid(row = 4,
                   columnspan = 3,
                   sticky = "ew"
                   )

  # Notebook to contain the two tables in our database
  table_frame = tk.Frame(home_view)
  table_frame.grid(row = 1,
                   column = 0,
                   sticky = "nsew"
                   )

  # Each column must take equal amount of space
  table_frame.rowconfigure(0, weight = 1)
  table_frame.rowconfigure(1, weight = 1)
  table_frame.rowconfigure(2, weight = 1)

  tables_notebook = ttk.Notebook(table_frame)
  tables_notebook.grid(row = 1,
                       padx = 10
                       )

  inventory_tab =  tk.Frame(tables_notebook)
  tables_notebook.add(inventory_tab, text =  "    Inventory    ")

  orders_tab =  tk.Frame(tables_notebook)
  tables_notebook.add(orders_tab, text =  "    Orders    ")

  populate_data() # populates data in the above notebook

  # functions for modelling view GUI
  def switch_to_home_view():
    home_view.tkraise()
```

```
def switch_to_inventory_models_view():
  inventory_models_view.tkraise()

def switch_to_order_models_view():
  order_models_view.tkraise()

def set_inventory_response_message(new_message):
  previous_text = inventory_response_message.get()
  new_text = ""
  if previous_text == "       Action status will be displayed here:       \n\n":
    # ie. response msg board is empty
    new_text = previous_text[:-1] + "       ▪ " + new_message + " ▪ \n"
  elif previous_text.count(" ▪ ") == 2:
    # if one msg already in msg board
    new_text = previous_text + "       ▪ " + new_message + " ▪ "
  elif previous_text.count(" ▪ ") == 4:
    # message board full
    messages = previous_text.split(" ▪ ")
    messages[1] = messages[3]
    messages[3] =  new_message
    new_text = " ▪ ".join(messages)

  inventory_response_message.set(new_text)

def set_orders_response_message(new_message):
  previous_text = orders_response_message.get()
  new_text = ""
  if previous_text == "       Action status will be displayed here:       \n\n":
    # ie. response msg board is empty
    new_text = previous_text[:-1] + "       ▪ " + new_message + " ▪ \n"
  elif previous_text.count(" ▪ ") == 2:
    # if one msg already in msg board
    new_text = previous_text + "       ▪ " + new_message + " ▪ "
  elif previous_text.count(" ▪ ") == 4:
    # message board full
    messages = previous_text.split(" ▪ ")
    messages[1] = messages[3]
    messages[3] =  new_message
    new_text = " ▪ ".join(messages)

  orders_response_message.set(new_text)

def close_plot():
  set_inventory_response_message("All open plots closed")
  set_orders_response_message("All open plots closed")
  plt.close()

def draw_plot(use_orders_table = False):
```

```
    #first, remove existing plot
    close_plot()

    if use_orders_table == False:
      x_axis = x_axis_column_name.get()
      y_axis = y_axis_column_name.get()
      z_axis = z_axis_column_name.get()
    else:
      x_axis = orders_x_axis_column_name.get()
      y_axis = orders_y_axis_column_name.get()
      z_axis = orders_z_axis_column_name.get()

    # without z-axis
    if x_axis != "unspecified" and y_axis != "unspecified" and z_axis == "unspecified":
      if use_orders_table == False:
        retrieved_data = database_handler.retrieve_via_sql_query(str(x_axis + "," + y_axis), "inventory")
      else:
        retrieved_data = database_handler.retrieve_via_sql_query(str(x_axis + "," + y_axis), "orders")
      x_axis_list = [retrieved_data[i][0] for i in range(0,len(retrieved_data))]
      y_axis_list = [retrieved_data[i][1] for i in range(0,len(retrieved_data))]

      ax = plt.axes()
      ax.plot(x_axis_list, y_axis_list, marker = 'x')
      ax.set_title(x_axis + " vs " + y_axis)
      ax.set_xlabel(x_axis)
      ax.set_ylabel(y_axis)

      set_inventory_response_message("x-y Graph drawn")
      plt.show()
    # with z-axis
    elif x_axis != "unspecified" and y_axis != "unspecified" and z_axis != "unspecified":
      if use_orders_table == False:
        retrieved_data = database_handler.retrieve_via_sql_query(str(x_axis + "," + y_axis + "," + z_axis), "inve
ntory")
      else:
        retrieved_data = database_handler.retrieve_via_sql_query(str(x_axis + "," + y_axis + "," + z_axis), "orde
rs")

      x_axis_list = [retrieved_data[i][0] for i in range(0,len(retrieved_data))]
      y_axis_list = [retrieved_data[i][1] for i in range(0,len(retrieved_data))]
      z_axis_list = [retrieved_data[i][2] for i in range(0,len(retrieved_data))]

      # checking if type is string as need to treat them diffenently in plot
      x_axis_type_string = False
      y_axis_type_string = False
      z_axis_type_string = False

      if isinstance(x_axis_list[0], int) == False and isinstance(x_axis_list[0], float) == False:
```

```
        # to check if string/date
        x_axis_type_string = True
        x_plot_data = range(len(x_axis_list))
      else:
        x_plot_data = x_axis_list

      if isinstance(y_axis_list[0], int) == False and isinstance(y_axis_list[0], float) == False:
        #to check if string/date
        y_axis_type_string = True
        y_plot_data = range(len(y_axis_list))
      else:
        y_plot_data = y_axis_list

      if isinstance(z_axis_list[0], int) == False and isinstance(z_axis_list[0], float) == False:
        #to check if string/date
        z_axis_type_string = True
        z_plot_data = range(len(z_axis_list))
      else:
        z_plot_data = z_axis_list

      ax = plt.axes(projection='3d')

      if plot_type.get()[17:] == "Scatter":
        set_orders_response_message("x-y-z Scatter Graph drawn")
        ax.scatter(x_plot_data, y_plot_data, z_plot_data, c= range(len(z_axis_list)), cmap='plasma', marker='x')
# colours reqiure numeric data always
      if plot_type.get()[17:] == "Line":
        set_orders_response_message("x-y-z Line Graph drawn")
        ax.plot3D(x_plot_data, y_plot_data, z_plot_data)

      # replacing the pseudo-
numbers (for string data) in the above statement by actual data if present (required to avoid datatype issu
es)
      if x_axis_type_string == True:
        ax.set(xticks=range(len(x_axis_list)), xticklabels=x_axis_list)

      if y_axis_type_string == True:
        ax.set(yticks=range(len(y_axis_list)), yticklabels=y_axis_list)

      if z_axis_type_string == True:
        ax.set(zticks=range(len(z_axis_list)), zticklabels=z_axis_list)

      ax.set_title(x_axis + " vs " + y_axis + " vs " + z_axis)
      ax.set_xlabel(x_axis, labelpad=20)
      ax.set_ylabel(y_axis, labelpad=20)
      ax.set_zlabel(z_axis, labelpad=20)

      plt.show()
```

```
def toggle_plot_type():
  plot_type_list = ["Scatter","Line"]
  current_index = plot_type_list.index(plot_type.get()[17:])

  current_index += 1
  current_index = current_index % len(plot_type_list)


  if current_index == 0:
    set_inventory_response_message("Graph type set to Scatter")
    set_orders_response_message("Graph type set to Scatter")
  else:
    set_inventory_response_message("Graph type set to Line")
    set_orders_response_message("Graph type set to Line")

  plot_type.set("📈 3D Graph type: " + plot_type_list[current_index])

def set_x_axis(use_orders_table = False):
  # retriving the column list
  if use_orders_table == False:
    table_information = database_handler.retrieve_headers("inventory")

    column_list = []
    for i in range(0,len(table_information)):
      column_list.append(table_information[i][0])

    # finding current index
    current_column = x_axis_column_name.get()
    try:
      current_column_index = column_list.index(current_column)
    except:
      #if unspecified, just sets it to -1 then +1 while displaying so sets to the zeroth position)
      current_column_index = -1

    #in case at the last column
    if current_column_index + 1 == len(column_list):
      current_column_index = -1

    x_axis_column_name.set(column_list[current_column_index + 1])

    #formating the text
    if len(x_axis_column_name.get()) < 25:
      x_axis.set("Set Graph's X-Axis:\n" + x_axis_column_name.get())
    else:
      x_axis.set("Set Graph's X-Axis:\n" + x_axis_column_name.get()[0:22] + "...")
  else:
    # use orders information
```

```
    table_information = database_handler.retrieve_headers("orders")

    column_list = []
    for i in range(0,len(table_information)):
      column_list.append(table_information[i][0])

    # finding current index
    current_column = orders_x_axis_column_name.get()
    try:
      current_column_index = column_list.index(current_column)
    except:
      #if unspecified, just sets it to -1 then +1 while displaying so sets to the zeroth position)
      current_column_index = -1

    #in case at the last column
    if current_column_index + 1 == len(column_list):
      current_column_index = -1

    orders_x_axis_column_name.set(column_list[current_column_index + 1])

    #formating the text
    if len(orders_x_axis_column_name.get()) < 25:
      orders_x_axis.set("Set Graph's X-Axis:\n" + orders_x_axis_column_name.get())
    else:
      orders_x_axis.set("Set Graph's X-Axis:\n" + orders_x_axis_column_name.get()[0:22] + "...")

def set_y_axis(use_orders_table = False):
  # retriving the column list
  if use_orders_table == False:
    table_information = database_handler.retrieve_headers("inventory")
    column_list = []
    for i in range(0,len(table_information)):
      column_list.append(table_information[i][0])

    # finding current index
    current_column = y_axis_column_name.get()
    try:
      current_column_index = column_list.index(current_column)
    except:
      #if unspecified, just sets it to -1 then +1 while displaying so sets to the zeroth position)
      current_column_index = -1

    #in case at the last column
    if current_column_index + 1 == len(column_list):
      current_column_index = -1

    y_axis_column_name.set(column_list[current_column_index + 1])
```

```
    #formating the text
    if len(y_axis_column_name.get()) < 25:
      y_axis.set("Set Graph's Y-Axis:\n" + y_axis_column_name.get())
    else:
      y_axis.set("Set Graph's Y-Axis:\n" + y_axis_column_name.get()[0:22] + "...")
  else:
    table_information = database_handler.retrieve_headers("orders")
    column_list = []
    for i in range(0,len(table_information)):
      column_list.append(table_information[i][0])

    # finding current index
    current_column = orders_y_axis_column_name.get()
    try:
      current_column_index = column_list.index(current_column)
    except:
      #if unspecified, just sets it to -1 then +1 while displaying so sets to the zeroth position)
      current_column_index = -1

    #in case at the last column
    if current_column_index + 1 == len(column_list):
      current_column_index = -1

    orders_y_axis_column_name.set(column_list[current_column_index + 1])

    #formating the text
    if len(orders_y_axis_column_name.get()) < 25:
      orders_y_axis.set("Set Graph's Y-Axis:\n" + orders_y_axis_column_name.get())
    else:
      orders_y_axis.set("Set Graph's Y-Axis:\n" + orders_y_axis_column_name.get()[0:22] + "...")

def set_z_axis(use_orders_table = False):
  if use_orders_table == False:
    # retriving the column list
    table_information = database_handler.retrieve_headers("inventory")
    column_list = []
    for i in range(0,len(table_information)):
      column_list.append(table_information[i][0])

    #as 3d plotting is optional,
    column_list.append("unspecified")

    # finding current index
    current_column = z_axis_column_name.get()
    try:
      current_column_index = column_list.index(current_column)
    except:
      #if unspecified, just sets it to -1 then +1 while displaying so sets to the zeroth position)
```

```
      current_column_index = -1

    #in case at the last column
    if current_column_index + 1 == len(column_list):
      current_column_index = -1

    z_axis_column_name.set(column_list[current_column_index + 1])

    #formating the text
    if len(z_axis_column_name.get()) < 25:
      z_axis.set("Set Graph's Z-Axis:\n" + z_axis_column_name.get())
    else:
      z_axis.set("Set Graph's Z-Axis:\n" + z_axis_column_name.get()[0:22] + "...")
  else:
    # retriving the column list
    table_information = database_handler.retrieve_headers("orders")
    column_list = []
    for i in range(0,len(table_information)):
      column_list.append(table_information[i][0])

    #as 3d plotting is optional,
    column_list.append("unspecified")

    # finding current index
    current_column = orders_z_axis_column_name.get()
    try:
      current_column_index = column_list.index(current_column)
    except:
      #if unspecified, just sets it to -1 then +1 while displaying so sets to the zeroth position)
      current_column_index = -1

    #in case at the last column
    if current_column_index + 1 == len(column_list):
      current_column_index = -1

    orders_z_axis_column_name.set(column_list[current_column_index + 1])

    #formating the text
    if len(orders_z_axis_column_name.get()) < 25:
      orders_z_axis.set("Set Graph's Z-Axis:\n" + orders_z_axis_column_name.get())
    else:
      orders_z_axis.set("Set Graph's Z-Axis:\n" + orders_z_axis_column_name.get()[0:22] + "...")

def tabularise_full_inventory_database(use_orders_table = False):
  if use_orders_table == False:
    # Create a new window
    full_inventory_database_window = tk.Toplevel(inventory_models_view)
    full_inventory_database_window.title("Full inventory viewer")
```

```
    full_inventory_database_window.geometry("1757x360")
    full_inventory_database_window.resizable(False, False)

    # Get header list
    headers = database_handler.retrieve_headers("inventory")
    column_list = []
    for i in range(0,len(headers)):
      header_name = headers[i][0][5:]
      if "manufacturer_" in header_name:
        header_name = "Manf. " + header_name[13:]
      column_list.append(header_name)

    full_data = database_handler.retrieve_via_sql_query("*","inventory")

    full_inventory_database_table = Treeview(full_inventory_database_window,
                          columns = column_list,
                          show = 'headings',
                          height = 17,
                          bootstyle = 'success'
                          )
    full_inventory_database_table.grid(row = 0,
                       column = 0,
                       sticky = "nsew"
                       )

    # creating the scrollbar
    scrollbar = ttk.Scrollbar(full_inventory_database_window, orient = "vertical", command = full_inventory
_database_table.yview)
    scrollbar.grid(row = 0,
                column = 1,
                sticky = "nsew"
                )
    full_inventory_database_table.configure(yscrollcommand = scrollbar.set)

    # initialising columns
    for i in column_list:
      full_inventory_database_table.column(i, anchor = "center", width = 145)
      full_inventory_database_table.heading(i, text = i)

    # insert values into full_inventory_database_window
    for i in full_data:
      full_inventory_database_table.insert(parent = '', index = tk.END, values = i)

    set_inventory_response_message("Opened full inventory viewer")
  else:
    # Create a new window
    full_orders_database_window = tk.Toplevel(order_models_view)
    full_orders_database_window.title("Full Orders viewer")
```

```
    full_orders_database_window.geometry("1465x360")
    full_orders_database_window.resizable(False, False)

    # Get header list
    headers = database_handler.retrieve_headers("orders")
    column_list = []
    for i in range(0,len(headers)):
      header_name = headers[i][0][6:]
      column_list.append(header_name)

    full_data = database_handler.retrieve_via_sql_query("*","orders")

    full_orders_database_table = Treeview(full_orders_database_window,
                          columns = column_list,
                          show = 'headings',
                          height = 17,
                          bootstyle = 'success'
                          )
    full_orders_database_table.grid(row = 0,
                        column = 0,
                        sticky = "nsew"
                        )

    # creating the scrollbar
    scrollbar = ttk.Scrollbar(full_orders_database_window, orient = "vertical", command = full_orders_data
base_table.yview)
    scrollbar.grid(row = 0,
                column = 1,
                sticky = "nsew"
                )
    full_orders_database_table.configure(yscrollcommand = scrollbar.set)

    # initialising columns
    for i in column_list:
      full_orders_database_table.column(i, anchor = "center", width = 145)
      full_orders_database_table.heading(i, text = i)

    # insert values into full_inventory_database_window
    for i in full_data:
      full_orders_database_table.insert(parent = '', index = tk.END, values = i)

    set_orders_response_message("Opened full orders viewer")

  def set_selected_item_name():
    # retriving the name list
    table_information = database_handler.retrieve_via_sql_query("item_id,item_name","inventory")

    # finding current index
```

```
  current_column_index = int(selected_item_id.get()[8:])

  # setting to name at the id
  set_inventory_response_message("Item set to: " + table_information[current_column_index][1] )
  selected_item_name.set("Item name: " + table_information[current_column_index][1])

def select_previous_item():
  # retriving the name list
  table_information = database_handler.retrieve_via_sql_query("item_id,item_name","inventory")

  # finding current index
  current_column_index = int(selected_item_id.get()[8:])

  #in case at the first column
  if current_column_index == 0:
    current_column_index = len(table_information) - 1
  else:
    current_column_index = current_column_index - 1

  selected_item_id.set("Item id: " + str(table_information[current_column_index][0]-1))
  set_selected_item_name()
  set_basic_inventory_turnover()
  set_reorder_warning()

def select_next_item():
  # retriving the name list
  table_information = database_handler.retrieve_via_sql_query("item_id,item_name","inventory")

  # finding current index
  current_column_index = int(selected_item_id.get()[8:])

  # in case at the last column
  if current_column_index + 1 == len(table_information):
    current_column_index = 0
  else:
    current_column_index = current_column_index + 1

  selected_item_id.set("Item id: " + str(table_information[current_column_index][0]-1))
  set_selected_item_name()
  set_basic_inventory_turnover()
  set_reorder_warning()

def set_selected_order_name():
  # retriving the name list
  table_information = database_handler.retrieve_via_sql_query("order_id,order_customer_name","orders")

  # finding current index
  current_column_index = int(selected_order_id.get()[10:])
```

```
  # setting to name at the id
  set_orders_response_message("Order id set to: " + str(table_information[current_column_index][0]-1))
  selected_order_name.set("Customer name: " + table_information[current_column_index][1])

def select_previous_order():
  # retriving the name list
  table_information = database_handler.retrieve_via_sql_query("order_id,order_customer_name","orders")
  # finding current index
  current_column_index = int(selected_order_id.get()[10:])

  #in case at the first column
  if current_column_index == 0:
    current_column_index = len(table_information) - 1
  else:
    current_column_index = current_column_index - 1

  selected_order_id.set("Order id: " + str(table_information[current_column_index][0]-1))
  set_selected_order_name()
  set_customer_spend_value()
  set_order_customer_number()

def select_next_order():
  # retriving the name list
  table_information = database_handler.retrieve_via_sql_query("order_id","orders")

  # finding current index
  current_column_index = int(selected_order_id.get()[10:])

  # in case at the last column
  if current_column_index + 1 == len(table_information):
    current_column_index = 0
  else:
    current_column_index = current_column_index + 1

  selected_order_id.set("Order id: " + str(table_information[current_column_index][0]-1))
  set_selected_order_name()
  set_customer_spend_value()
  set_order_customer_number()


# modelling view GUI
title_row = tk.Frame(modelling_view)
title_row.columnconfigure(0, weight = 1) # centering
title_row.columnconfigure(1, weight = 0)
title_row.columnconfigure(2, weight = 1)
title_row.grid(row = 0,
               columnspan = 3,
```

```
            sticky = "nsew"
            )


 inventory_view_button = Button(title_row,
                        text = "Inventory",
                        command = switch_to_inventory_models_view,
                        bootstyle = "warning-outline"
                        )
 inventory_view_button.grid(row = 0,
                      column = 0
                      )

 order_view_button = Button(title_row,
                      text = "Orders",
                      command = switch_to_order_models_view,
                      bootstyle = "success-outline"
                      )
 order_view_button.grid(row = 0,
                   column = 1
                   )


 title_label = tk.Label(title_row, text = "       ▭▭ ▪ ▣▓ ▓ ▒ Modelling View ▒ ▓ ▓▣ ▪ ▭▭       ", relief
= "ridge", font = "TkFixedFont")
 title_label.grid(row = 0,
               column = 2
               )

 back_button = Button(title_row,
                  text = "Back",
                  command = switch_to_home_view,
                  bootstyle = "primary-outline"
                  )
 back_button.grid(row = 0,
               column = 3,
               pady = 5
               )

 inventory_models_view = tk.Frame(modelling_view)
 inventory_models_view.grid(row = 1,
                      columnspan = 2,
                      sticky = "nsew"
                      )

 order_models_view = tk.Frame(modelling_view)
 order_models_view.grid(row = 1,
                   columnspan = 2,
```

```
                    sticky = "nsew"
                    )

  # specific functions for inventory view
  def get_selected_item_values():
    selected_item_values = database_handler.retrieve_via_sql_query("item_cost,item_margin,item_stock,item
_restock_value","inventory")
    return(selected_item_values)

  def set_basic_inventory_turnover():
      selected_item_values = get_selected_item_values()
      # ^ in format  [(item_name,item_cost,item_margin,item_stock), so on...]

      current_column_index = int(selected_item_id.get()[8:])

      cost = selected_item_values[current_column_index][0]
      margin = selected_item_values[current_column_index][1]
      stock = selected_item_values[current_column_index][2]

      if stock == 0:
          basic_inventory_turnover_value.set("Basic inventory Turnover: 0.0")
      else:
        basic_inventory_turnover_value.set("Basic inventory Turnover: " + str(round(((cost * (100 -
 margin)) / stock),6)))

  def set_reorder_warning():
      selected_item_values = get_selected_item_values()
      # ^ in format  [(item_name,item_cost,item_margin,item_stock), so on...]

      current_column_index = int(selected_item_id.get()[8:])

      stock = selected_item_values[current_column_index][2]

      if stock < 100:
          reorder_warning_value.set("Reorder point reached: True")
      else:
        reorder_warning_value.set("Reorder point reached: False")

  def populate_inventory_low_stocks_data():
    inventory_low_stocks_viewer_frame = tk.Frame(inventory_models_view)
    inventory_low_stocks_viewer_frame.grid(row = 1,
                                column = 2,
                                rowspan = 6,
                                sticky = "e"
                                )

    inventory_low_stocks_data = database_handler.retrieve_via_sql_query("item_name,item_stock,item_resto
ck_value","inventory")
```

```
    inventory_low_stocks_viewer = Treeview(inventory_low_stocks_viewer_frame,
                                    columns = ("item_name","item_stock_restock_value_difference"),
                                    show = 'headings',
                                    height = 13,
                                    bootstyle = 'success'
                                    )
    inventory_low_stocks_viewer.grid(row = 1,
                                column = 1
                                )

    reformatted_data = []
    temp_inventory_low_stocks_data = inventory_low_stocks_data
    difference_list = []
    for i in inventory_low_stocks_data:
      # inventory_low_stocks_data in format [(item_name,item_stock,item_restock_value)...so on]
      difference_list.append(i[1]-i[2]) # append the difference between stock and restock value

    while len(difference_list) != 0:
      minimum_difference  = min(difference_list)
      minimum_difference_index = difference_list.index(minimum_difference)

      minimum_difference_data = temp_inventory_low_stocks_data[minimum_difference_index]
      reformatted_data.append((minimum_difference_data[0],minimum_difference_data[1]-
minimum_difference_data[2])) #appends a tuple in format (name,difference)

      temp_inventory_low_stocks_data.pop(minimum_difference_index)
      difference_list.pop(minimum_difference_index)

    # creating the scrollbar
    inventory_low_stocks_scrollbar = ttk.Scrollbar(inventory_low_stocks_viewer_frame, orient = "vertical", co
mmand = inventory_low_stocks_viewer.yview)
    inventory_low_stocks_scrollbar.grid(row = 1,
                                column = 2,
                                sticky = "nsew"
                                )
    inventory_low_stocks_viewer.configure(yscrollcommand = inventory_low_stocks_scrollbar.set)

    # initialising columns
    inventory_low_stocks_viewer.column("item_name", anchor = "center", width = 85)
    inventory_low_stocks_viewer.heading('item_name', text = 'Name')
    inventory_low_stocks_viewer.column("item_stock_restock_value_difference", anchor = "center", width =
55)
    inventory_low_stocks_viewer.heading('item_stock_restock_value_difference', text = "Diff.")

    # insert values into inventory_low_stocks_viewer
    for i in reformatted_data:
      inventory_low_stocks_viewer.insert(parent = '', index = tk.END, values = i)
```

```
 # inventory column 1
 graph_plotter_label = tk.Label(inventory_models_view, text = "  ▪ ▣▓ ▩ ▒ Graph Plotter ▒ ▩ ▓▣ ▪  ", r
elief = "ridge")
 graph_plotter_label.grid(row = 0,
                          column = 0
                          )

 x_axis = tk.StringVar()
 x_axis_column_name = tk.StringVar()
 set_x_axis_button = tk.Button(inventory_models_view,
                               textvariable = x_axis,
                               command = set_x_axis
                               )
 set_x_axis_button.grid(row = 1,
                        column = 0,
                        pady = 2,
                        sticky = "nsew"
                        )
 x_axis_column_name.set("unspecified")
 x_axis.set("Set Graph's X-Axis:\n" + x_axis_column_name.get())

 y_axis = tk.StringVar()
 y_axis_column_name = tk.StringVar()
 set_y_axis_button = tk.Button(inventory_models_view,
                               textvariable = y_axis,
                               command = set_y_axis
                               )
 set_y_axis_button.grid(row = 2,
                        column = 0,
                        pady = 2,
                        sticky = "nsew"
                        )
 y_axis_column_name.set("unspecified")
 y_axis.set("Set Graph's Y-Axis:\n" + y_axis_column_name.get())

 z_axis = tk.StringVar()
 z_axis_column_name = tk.StringVar()
 set_z_axis_button = tk.Button(inventory_models_view,
                               textvariable = z_axis,
                               command = set_z_axis
                               )
 set_z_axis_button.grid(row = 3,
                        column = 0,
                        pady = 2,
                        sticky = "nsew"
                        )
 z_axis_column_name.set("unspecified")
 z_axis.set("Set Graph's Z-Axis:\n" + z_axis_column_name.get())
```

```
plot_button_frame = tk.Frame(inventory_models_view)
plot_button_frame.grid(row = 4,
                       column = 0,
                       sticky = "nsew"
                       )

scatter_plot_button = Button(plot_button_frame,
                         text = " 📊 Plot Graph",
                         command = draw_plot,
                         bootstyle = "warning-outline"
                         )
scatter_plot_button.grid(row = 0,
                     column = 0,
                     padx = 2,
                     pady = 5
                     )

graph_close_button = Button(plot_button_frame,
                         text = "Close Graph",
                         command = close_plot,
                         bootstyle = "success-outline"
                         )
graph_close_button.grid(row = 0,
                      column = 1,
                      padx = 2,
                      pady = 5
                      )

plot_type = tk.StringVar()
plot_type_button = tk.Button(inventory_models_view,
                         textvariable = plot_type,
                         command = toggle_plot_type
                         )
plot_type_button.grid(row = 5,
                     column = 0,
                     sticky = "nsew"
                     )
plot_type.set("📈 3D Graph type: Scatter")

date_frame = tk.Frame(inventory_models_view)
date_frame.grid(row = 6,
                column = 0,
                pady = 5,
                sticky = "nsew",
                )

day_month_year_label = tk.Label(date_frame, text = " Date | Month | Year ", relief = "groove")
```

```
 day_month_year_label.grid(row = 0,
                           columnspan = 3,
                           pady = 2,
                           sticky = "nsew"
                           )

 current_date = datetime.date.today()
 day_label = tk.Label(date_frame,
                      text = str(current_date)[8:],
                      relief = "raised",
                      width = 8
                      )
 day_label.grid(row = 1,
                column = 0,
                padx = 1,
                sticky = "nsew"
                )

 month_label = tk.Label(date_frame,
                        text = str(current_date)[5:7],
                        relief = "raised",
                        width = 8
                        )
 month_label.grid(row = 1,
                  column = 1,
                  padx = 4,
                  sticky = "nsew"
                  )

 year_label = tk.Label(date_frame,
                       text = str(current_date)[0:4],
                       relief = "raised",
                       width = 8
                       )
 year_label.grid(row = 1,
                 column = 2,
                 padx = 1,
                 sticky = "nsew"
                 )

 # inventory column 2
 inventory_response_message = tk.StringVar()
 inventory_response_message_board = Label(inventory_models_view, textvariable = inventory_response_
message, bootstyle = "inverse-dark")
 inventory_response_message_board.grid(row = 0,
                                       column = 1,
                                       padx = 3,
                                       pady = 2,
```

```
                                rowspan = 2,
                                sticky = "nsew"
                                )
inventory_response_message.set("       Action status will be displayed here:       \n\n")

full_inventory_database_viewer_button = Button(inventory_models_view,
                                    text = "View full inventory database",
                                    command = tabularise_full_inventory_database,
                                    bootstyle = "warning"
                                    )
full_inventory_database_viewer_button.grid(row = 2,
                                column = 1,
                                padx = 3,
                                pady = 2,
                                sticky = "nsew"
                                )

itemwise_statistics_frame = tk.Frame(inventory_models_view)
itemwise_statistics_frame.grid(row = 3,
                        column = 1
                        )

left_spacer = tk.Label(itemwise_statistics_frame, text = "     ", relief = "groove")
left_spacer.grid(row = 0,
            column = 0,
            padx = 2
            )

itemwise_statistics_label = tk.Label(itemwise_statistics_frame,
                            text = " ▪ ▣▒ Itemwise Statistics ▒▣ ▪ ",
                            relief = "groove"
                            )
itemwise_statistics_label.grid(row = 0,
                        column = 1
                        )

right_spacer = tk.Label(itemwise_statistics_frame, text = "     ", relief = "groove")
right_spacer.grid(row = 0,
            column = 2,
            padx = 2
            )

previous_item_button = Button(itemwise_statistics_frame,
                        text = "<",
                        command = select_previous_item,
                        bootstyle = "success"
                        )
previous_item_button.grid(row = 1,
```

```
                    column = 0,
                    padx = 2
                    )

selected_item_id = tk.StringVar()
selected_item_label = tk.Label(itemwise_statistics_frame,
                        textvariable = selected_item_id,
                        relief = "groove"
                        )
selected_item_id.set("Item id: 0")
selected_item_label.grid(row = 1,
                  column = 1,
                  sticky = "nsew"
                  )

next_item_button = Button(itemwise_statistics_frame,
                      text = ">",
                      command = select_next_item,
                      bootstyle = "success"
                      )
next_item_button.grid(row = 1,
                column = 2,
                padx = 2
                )

selected_item_name = tk.StringVar()
selected_item_name_label = tk.Label(inventory_models_view,
                        textvariable = selected_item_name,
                        relief = "groove"
                        )
set_selected_item_name()
selected_item_name_label.grid(row = 4,
                        column = 1,
                        pady = 5,
                        padx = 2,
                        sticky = "nsew"
                        )

basic_inventory_turnover_value = tk.StringVar()
basic_inventory_turnover_label = Label(inventory_models_view,
                              textvariable = basic_inventory_turnover_value,
                              bootstyle = "inverse-dark")
set_basic_inventory_turnover()
basic_inventory_turnover_label.grid(row = 5,
                            column = 1,
                            pady = 1,
                            padx = 3,
                            sticky = "nsew"
```

```
                                )

  reorder_warning_value = tk.StringVar()
  reorder_warning_label = Label(inventory_models_view,
                                textvariable = reorder_warning_value,
                                bootstyle = "inverse-dark")
  set_reorder_warning()
  reorder_warning_label.grid(row = 6,
                             column = 1,
                             pady = 7,
                             padx = 3,
                             sticky = "nsew"
                             )

  # inventory column 3
  inventory_low_stocks_label = tk.Label(inventory_models_view, text = "▣ Stocks Till Restock ▣", relief = "ri
dge")
  inventory_low_stocks_label.grid(row = 0,
                            column = 2,
                            sticky = "nsewe"
                            )
  populate_inventory_low_stocks_data()

  # wrapper functions for orders view
  def orders_set_x_axis():
    set_x_axis(use_orders_table = True)

  def orders_set_y_axis():
    set_y_axis(use_orders_table = True)

  def orders_set_z_axis():
    set_z_axis(use_orders_table = True)

  def orders_draw_plot():
    draw_plot(use_orders_table = True)

  def orders_tabularise_full_database():
    tabularise_full_inventory_database(use_orders_table = True)

  # specific functions for orders view
  def set_customer_spend_value():

    current_order_id = int(selected_order_id.get()[9:]) + 1
    where_clause = "order_id =" + str(current_order_id)

    retrieved_data = database_handler.retrieve_via_sql_query("order_id,order_final_cost,order_quantity","ord
ers",where_clause)
```

```
    #retrieved_data in format [(order_id,order_final_cost,order_quantity)]
    customer_spend_value.set("Total amt: " + str(retrieved_data[0][1]*retrieved_data[0][2]))

  def set_order_customer_number():
    current_order_id = int(selected_order_id.get()[9:]) + 1
    where_clause = "order_id =" + str(current_order_id)

    retrieved_data = database_handler.retrieve_via_sql_query("order_id,order_customer_contact_no","orders
",where_clause)

    #retrieved_data in format [(order_id,order_customer_contact_no)]
    try:
      order_customer_number.set("Contact Number: " + str(retrieved_data[0][1]))
    except:
      # incase the customer didn't give their phone number
      order_customer_number.set("Contact Number: Not Available")

  def populate_orders_highest_spends_data():
    orders_highest_spends_viewer_frame = tk.Frame(order_models_view)
    orders_highest_spends_viewer_frame.grid(row = 1,
                                            column = 2,
                                            rowspan = 6,
                                            sticky = "e"
                                            )

    orders_highest_spends_data = database_handler.retrieve_via_sql_query("order_customer_name,sum(ord
er_final_cost*order_quantity) as total_spend","orders",sql_group_by = "order_customer_name")
    orders_highest_spends_viewer = Treeview(orders_highest_spends_viewer_frame,
                                            columns = ("order_customer_name","total_spend"),
                                            show = 'headings',
                                            height = 13,
                                            bootstyle = 'success'
                                            )
    orders_highest_spends_viewer.grid(row = 1,
                                      column = 1
                                      )

    orders_reformatted_data = []
    temp_orders_highest_spends_data = orders_highest_spends_data
    total_spends_list = []
    for i in orders_highest_spends_data:
      # orders_highest_spends_data in format [(order_customer_name,total_spend)...so on]
      total_spends_list.append(i[1]) # append the total spend

    while len(total_spends_list) != 0:
      maximum_spend_index = total_spends_list.index(max(total_spends_list))

      maximum_spend_data = temp_orders_highest_spends_data[maximum_spend_index]
```

```
    orders_reformatted_data.append((maximum_spend_data[0],maximum_spend_data[1])) #appends a tup
le in format (name,total_spend)

    temp_orders_highest_spends_data.pop(maximum_spend_index)
    total_spends_list.pop(maximum_spend_index)
  # creating the scrollbar
  orders_highest_spends_scrollbar = ttk.Scrollbar(orders_highest_spends_viewer_frame, orient = "vertical",
 command = orders_highest_spends_viewer.yview)
  orders_highest_spends_scrollbar.grid(row = 1,
                               column = 2,
                               sticky = "nsew"
                               )
  orders_highest_spends_viewer.configure(yscrollcommand = orders_highest_spends_scrollbar.set)

  # initialising columns
  orders_highest_spends_viewer.column("order_customer_name", anchor = "center", width = 75)
  orders_highest_spends_viewer.heading('order_customer_name', text = 'Name')
  orders_highest_spends_viewer.column("total_spend", anchor = "center", width = 65)
  orders_highest_spends_viewer.heading('total_spend', text = "Total")

  # insert values into orders_highest_spends_viewer
  for i in orders_reformatted_data:
    orders_highest_spends_viewer.insert(parent = '', index = tk.END, values = i)

 # orders column 1
 orders_graph_plotter_label = tk.Label(order_models_view, text = "  ▪ ▣▓ ▩ ▒ Graph Plotter ▒ ▩ ▓▣ ▪
", relief = "ridge")
 orders_graph_plotter_label.grid(row = 0,
                     column = 0
                     )

 orders_x_axis = tk.StringVar()
 orders_x_axis_column_name = tk.StringVar()
 orders_set_x_axis_button = tk.Button(order_models_view,
                          textvariable = orders_x_axis,
                          command = orders_set_x_axis
                          )
 orders_set_x_axis_button.grid(row = 1,
                   column = 0,
                   pady = 2,
                   sticky = "nsew"
                   )
 orders_x_axis_column_name.set("unspecified")
 orders_x_axis.set("Set Graph's X-Axis:\n" + orders_x_axis_column_name.get())

 orders_y_axis = tk.StringVar()
 orders_y_axis_column_name = tk.StringVar()
 orders_set_y_axis_button = tk.Button(order_models_view,
```

```
                        textvariable = orders_y_axis,
                        command = orders_set_y_axis
                        )
orders_set_y_axis_button.grid(row = 2,
                column = 0,
                pady = 2,
                sticky = "nsew"
                )
orders_y_axis_column_name.set("unspecified")
orders_y_axis.set("Set Graph's Y-Axis:\n" + orders_y_axis_column_name.get())

orders_z_axis = tk.StringVar()
orders_z_axis_column_name = tk.StringVar()
orders_set_z_axis_button = tk.Button(order_models_view,
                        textvariable = orders_z_axis,
                        command = orders_set_z_axis
                        )
orders_set_z_axis_button.grid(row = 3,
                column = 0,
                pady = 2,
                sticky = "nsew"
                )
orders_z_axis_column_name.set("unspecified")
orders_z_axis.set("Set Graph's Z-Axis:\n" + orders_z_axis_column_name.get())

orders_plot_button_frame = tk.Frame(order_models_view)
orders_plot_button_frame.grid(row = 4,
                    column = 0,
                    sticky = "nsew"
                    )

orders_scatter_plot_button = Button(orders_plot_button_frame,
                        text = " 📊 Plot Graph",
                        command = orders_draw_plot,
                        bootstyle = "warning-outline"
                        )
orders_scatter_plot_button.grid(row = 0,
                column = 0,
                padx = 2,
                pady = 5
                )

orders_graph_close_button = Button(orders_plot_button_frame,
                        text = "Close Graph",
                        command = close_plot,
                        bootstyle = "success-outline"
                        )
orders_graph_close_button.grid(row = 0,
```

```
                    column = 1,
                    padx = 2,
                    pady = 5
                    )

  orders_plot_type = tk.StringVar()
  orders_plot_type_button = tk.Button(order_models_view,
                          textvariable = plot_type,
                          command = toggle_plot_type
                          )
  orders_plot_type_button.grid(row = 5,
                    column = 0,
                    sticky = "nsew"
                    )
  orders_plot_type.set(" 📈 3D Graph type: Scatter")

  orders_date_frame = tk.Frame(order_models_view)
  orders_date_frame.grid(row = 6,
                column = 0,
                pady = 5,
                sticky = "nsew",
                )

  orders_day_month_year_label = tk.Label(orders_date_frame, text = " Date | Month | Year ", relief = "groov
e")
  orders_day_month_year_label.grid(row = 0,
                      columnspan = 3,
                      pady = 2,
                      sticky = "nsew"
                      )

  orders_day_label = tk.Label(orders_date_frame,
                    text = str(current_date)[8:],
                    relief = "raised",
                    width = 8
                    )
  orders_day_label.grid(row = 1,
                column = 0,
                padx = 1,
                sticky = "nsew"
                )

  orders_month_label = tk.Label(orders_date_frame,
                      text = str(current_date)[5:7],
                      relief = "raised",
                      width = 8
                      )
  orders_month_label.grid(row = 1,
```

```
            column = 1,
            padx = 4,
            sticky = "nsew"
            )

 orders_year_label = tk.Label(orders_date_frame,
                text = str(current_date)[0:4],
                relief = "raised",
                width = 8
                )
 orders_year_label.grid(row = 1,
            column = 2,
            padx = 1,
            sticky = "nsew"
            )

 # orders column 2
 orders_response_message = tk.StringVar()
 orders_inventory_response_message_board = Label(order_models_view, textvariable = orders_response_
message, bootstyle = "inverse-dark")
 orders_inventory_response_message_board.grid(row = 0,
                            column = 1,
                            padx = 3,
                            pady = 2,
                            rowspan = 2,
                            sticky = "nsew"
                            )
 orders_response_message.set("      Action status will be displayed here:      \n\n")

 orders_full_inventory_database_viewer_button = Button(order_models_view,
                                text = "View full orders database",
                                command = orders_tabularise_full_database,
                                bootstyle = "success"
                                )
 orders_full_inventory_database_viewer_button.grid(row = 2,
                                column = 1,
                                padx = 3,
                                pady = 2,
                                sticky = "nsew"
                                )

 orderwise_statistics_frame = tk.Frame(order_models_view)
 orderwise_statistics_frame.grid(row = 3,
                        column = 1
                        )

 orders_left_spacer = tk.Label(orderwise_statistics_frame, text = "     ", relief = "groove")
 orders_left_spacer.grid(row = 0,
```

```
            column = 0,
            padx = 2
            )

orderwise_statistics_label = tk.Label(orderwise_statistics_frame,
                        text = " ▪ ▣▓ Orderwise Statistics ▓▣ ▪ ",
                        relief = "groove"
                        )
orderwise_statistics_label.grid(row = 0,
                      column = 1
                      )

orders_right_spacer = tk.Label(orderwise_statistics_frame, text = "     ", relief = "groove")
orders_right_spacer.grid(row = 0,
            column = 2,
            padx = 2
            )

orders_previous_item_button = Button(orderwise_statistics_frame,
                     text = "<",
                     command = select_previous_order,
                     bootstyle = "success"
                     )
orders_previous_item_button.grid(row = 1,
                    column = 0,
                    padx = 2
                    )

selected_order_id = tk.StringVar()
selected_order_label = tk.Label(orderwise_statistics_frame,
                       textvariable = selected_order_id,
                       relief = "groove"
                       )
selected_order_id.set("Order id: 0")
selected_order_label.grid(row = 1,
                  column = 1,
                  sticky = "nsew"
                  )

orders_next_item_button = Button(orderwise_statistics_frame,
                     text = ">",
                     command = select_next_order,
                     bootstyle = "success"
                     )
orders_next_item_button.grid(row = 1,
                 column = 2,
                 padx = 2
                 )
```

```
  selected_order_name = tk.StringVar()
  selected_order_name_label = tk.Label(order_models_view,
                           textvariable = selected_order_name,
                           relief = "groove"
                           )
  set_selected_order_name()
  selected_order_name_label.grid(row = 4,
                          column = 1,
                          pady = 5,
                          padx = 2,
                          sticky = "nsew"
                          )

  customer_spend_value = tk.StringVar()
  customer_spend_label = Label(order_models_view,
                               textvariable = customer_spend_value,
                               bootstyle = "inverse-dark")
  set_customer_spend_value()
  customer_spend_label.grid(row = 5,
                            column = 1,
                            pady = 1,
                            padx = 3,
                            sticky = "nsew"
                            )

  order_customer_number = tk.StringVar()
  order_customer_number_label = Label(order_models_view,
                               textvariable = order_customer_number,
                               bootstyle = "inverse-dark")
  set_order_customer_number()
  order_customer_number_label.grid(row = 6,
                            column = 1,
                            pady = 7,
                            padx = 3,
                            sticky = "nsew"
                            )

  # orders column 3
  inventory_low_stocks_label = tk.Label(order_models_view, text = " ▣ Highest Spenders ▣ ", relief = "ridg
e")
  inventory_low_stocks_label.grid(row = 0,
                           column = 2,
                           sticky = "nsew"
                           )
  populate_orders_highest_spends_data()

  inventory_models_view.tkraise()
```

```
# Sets initial frame to be home_view
home_view.tkraise()

root.mainloop()
```

Module Overview

The visual_handler.py module provides a Graphical User Interface (GUI) for an Inventory and Order Analytics System.

It integrates database operations, file import, and visualization to help users manage and analyze inventory/order datasets.

Key features:

- View inventory and orders in tabular format (Treeview with scrollbars).
- Import CSV data for inventory and orders into a database.
- Generate 2D and 3D graphs of database columns.
- Compute and display useful statistics (e.g., turnover, reorder warnings, customer spend).
- Navigate between Home View (database viewer) and Modelling View (analytics & visualization).
- Styled UI using TTK Bootstrap with modern themes.

Main Function

run_GUI()

Description: Entry point of the application.

Responsibilities:

- Initialize Tkinter root window (Tk() object).
- Configure Home and Modelling views.
- Create styled UI components using TTK Bootstrap.
- Define all nested functions for database operations, plotting, and navigation.
- Run the Tkinter event loop (mainloop).

Home View Functions

Function	Purpose	Key Details
`populate_data()`	Displays inventory and orders tables.	Uses `Treeview` with vertical scrollbars; fetches data via `database_handler.retrieve_via_sql_query()`.
`import_database()`	Imports CSV data for inventory and	Uses NumPy's `genfromtxt`. Calls `database_handler.import_items()` and

	orders.	`database_handler.import_orders()`. Handles errors with messages.
`set_inventory_path()` / `set_orders_path()`	Opens file dialogs to select CSV files.	Updates `StringVar` labels with truncated file paths for clarity.
`refresh()`	Refreshes database displays and model statistics.	Calls `populate_data()`, `populate_inventory_low_stocks_data()`, and `populate_orders_highest_spends_data()`.
`switch_to_modelling_view()`	Switches GUI to Modelling View.	Uses `tkraise()` to bring modelling frame to front.

Modelling View Functions

Function	Purpose	Key Details
`switch_to_home_view()`	Returns to Home View.	Uses `tkraise()`.
`switch_to_inventory_models_view()` / `switch_to_order_models_view()`	Switch between inventory/order analytics sections.	
`set_inventory_response_message()` / `set_orders_response_message()`	Updates status boards.	Maintains last 2 messages in a rotating buffer.
`close_plot()`	Closes all open Matplotlib plots.	Calls `plt.close()`.
`draw_plot(use_orders_table=False)`	Creates 2D/3D plots.	- Retrieves selected x, y, z columns. - Supports both string and numeric axes. - For 3D: supports Scatter and Line plots.
`toggle_plot_type()`	Switches between Scatter and Line plot modes.	Updates message boards and button label.
`set_x_axis()` / `set_y_axis()` / `set_z_axis()`	Cycles through available database columns for axes.	Maintains current column index and loops back if at end.
`tabularise_full_inventory_database(use_orders_table=False)`	Opens new window to display full database table.	- Uses `Treeview` with scrollbars. - Supports both inventory and orders.

Inventory-Specific Functions

Function	Purpose	Key Details

get_selected_item_values()	Retrieves selected item details.	Queries `item_cost`, `item_margin`, `item_stock`, `item_restock_value`.
set_basic_inventory_turnover()	Calculates turnover ratio.	Formula: `(cost × (100 - margin)) / stock`.
set_reorder_warning()	Displays reorder warning.	Alerts when `stock < 100`.
populate_inventory_low_stocks_data()	Lists items closest to restocking point.	Sorts by `(stock - restock_value)` difference.
set_selected_item_name()	Displays current item name.	Syncs with `selected_item_id`.
select_previous_item() / **select_next_item()**	Navigate inventory items sequentially.	Wraps around at list edges.

Order-Specific Functions

Function	Purpose	Key Details
set_selected_order_name()	Displays customer name for selected order.	Syncs with `selected_order_id`.
select_previous_order() / **select_next_order()**	Navigate through orders sequentially.	Wraps around at start/end.
set_customer_spend_value()	Calculates total spend for an order.	Formula: `order_final_cost × order_quantity`.
set_order_customer_number()	Displays customer contact number.	Handles missing phone numbers gracefully.
populate_orders_highest_spends_data()	Lists customers sorted by spend.	Aggregates data with `SUM(order_final_cost × quantity)` and sorts descending.
orders_set_x_axis() / **orders_set_y_axis()** / **orders_set_z_axis()**	Axis selection wrappers for order tables.	Calls generic axis functions with `use_orders_table=True`.
orders_draw_plot()	Plots order data.	Wrapper for `draw_plot()` with

		`use_orders_table=True`.
`orders_tabularise_full_database()`	Opens full orders table in new window.	Similar to inventory tabularization.

Reserved Words Used

The module makes extensive use of Python reserved keywords:

- Function definitions: def, return
- Control flow: if, elif, else, for, while
- Error handling: try, except
- Imports: import, from, as
- Booleans: True, False
- Special values: None
- Membership operators: in, not in

Libraries Used

Library	Purpose
tkinter	Core GUI framework (windows, frames, labels, buttons).
tkinter.ttk	Styled widgets (Notebook, Scrollbar, Treeview).
tkinter.filedialog.askopenfilename	File selection dialogs for importing CSVs.
numpy (genfromtxt)	CSV file parsing into structured arrays.
database_handler (custom)	Handles SQL queries and database imports.
matplotlib.pyplot	Plotting 2D/3D graphs for data analysis.
ttkbootstrap	Theming and styled widgets for Tkinter.
datetime	Retrieve and display current date.
sys	Exit application (`sys.exit`).

Usage Flow

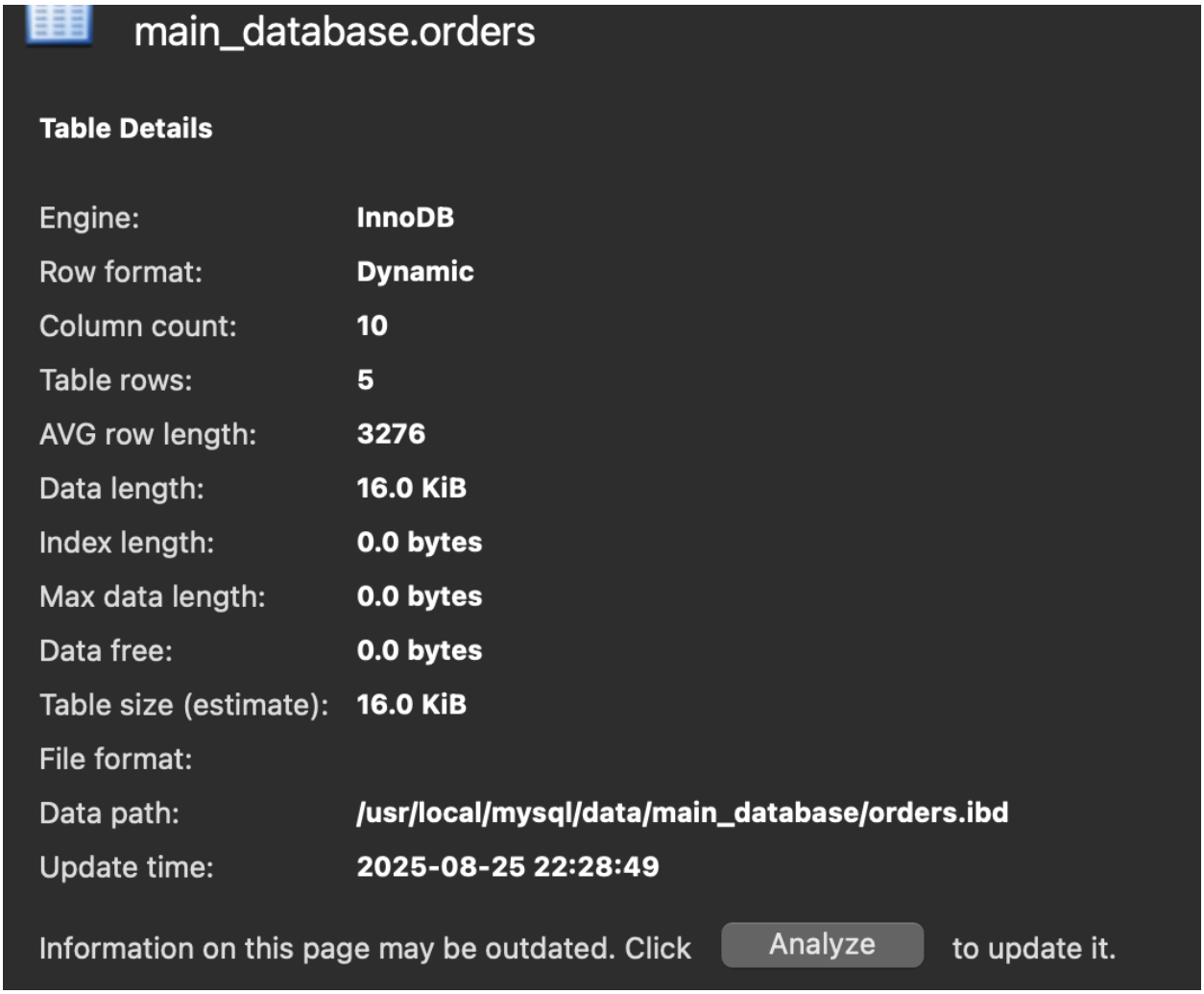
1. Launch GUI using run_GUI().
2. Home View:
    a. Import database CSVs.
    b. View inventory/orders in tabular format.
    c. Refresh data.
3. Switch to Modelling View:
    a. Select columns for x, y, z axes.
    b. Generate 2D/3D plots.
    c. View statistics like turnover, reorder warnings, highest spenders.

    d. Open full database tables in new windows.
4. Quit application via Exit button.

Full Keyword List (Python 3.12)

`False`, `None`, `True`, `and`, `as`, `assert`, `async`, `await`, `break`, `class`, `continue`, `def`, `del`, `elif`, `else`, `except`, `finally`, `for`, `from`, `global`, `if`, `import`, `in`, `is`, `lambda`, `match`, `nonlocal`, `not`, `or`, `pass`, `raise`, `return`, `try`, `while`, `with`, `yield`, `case`.

Database



Column	Type	Default Value	Nullable	Character Set	Collation
order_id	int		NO		
order_item_name	varchar(20)		NO	utf8mb4	utf8mb4_090...
order_date	date		NO		
order_initial_cost	float(8,2)		NO		
order_gst	float(8,2)		NO		
order_discount	float(5,2)		NO		
order_final_cost	float(8,2)		NO		
order_quantity	int		NO		
order_customer_n...	varchar(20)		NO	utf8mb4	utf8mb4_090...
order_customer_c...	varchar(10)		YES	utf8mb4	utf8mb4_090...

Indexes in Table

Visible	Key	Type	Uniq...	Columns
☑	PRIMARY	BTREE	YES	order_id

Index Details

Key Name:
Index Type:
Allows NULL:
Cardinality:
Comment:
User Comment:

Columns in table

Column	Type	Nullable	Indexes
order_id	int	NO	PRIMARY
order_item_name	varchar(20)	NO	
order_date	date	NO	
order_initial_cost	float(8,2)	NO	
order_gst	float(8,2)	NO	
order_discount	float(5,2)	NO	
order_final_cost	float(8,2)	NO	
order_quantity	int	NO	

main_database.inventory

Table Details

Engine:	**InnoDB**
Row format:	**Dynamic**
Column count:	**12**
Table rows:	**5**
AVG row length:	**3276**
Data length:	**16.0 KiB**
Index length:	**0.0 bytes**
Max data length:	**0.0 bytes**
Data free:	**0.0 bytes**
Table size (estimate):	**16.0 KiB**
File format:	
Data path:	**/usr/local/mysql/data/main_database/inventory.ibd**
Update time:	**2025-08-25 22:28:49**

Information on this page may be outdated. Click Analyze to update it.

Column	Type	Default Value	Nullable	Character Set	Collation
item_id	int		NO		
item_name	varchar(20)		NO	utf8mb4	utf8mb4_090...
item_cost	float(8,2)		NO		
item_gst	float(8,2)		NO		
item_discount	float(5,2)		NO		
item_final_cost	float(8,2)		NO		
item_margin	float(5,2)		NO		
item_stock	int		NO		
item_restock_value	int		NO		
item_manufacture...	varchar(40)		NO	utf8mb4	utf8mb4_090...
item_manufacture...	varchar(20)		NO	utf8mb4	utf8mb4_090...
item_manufacture...	varchar(10)		NO	utf8mb4	utf8mb4_090...

Indexes in Table

Visible	Key	Type	Uniq...	Columns
☑	PRIMARY	BTREE	YES	item_id

Index Details

Key Name:
Index Type:
Allows NULL:
Cardinality:
Comment:
User Comment:

Columns in table

Column	Type	Nullable	Indexes
item_id	int	NO	PRIMARY
item_name	varchar(20)	NO	
item_cost	float(8,2)	NO	
item_gst	float(8,2)	NO	
item_discount	float(5,2)	NO	
item_final_cost	float(8,2)	NO	
item_margin	float(5,2)	NO	
item_stock	int	NO	